

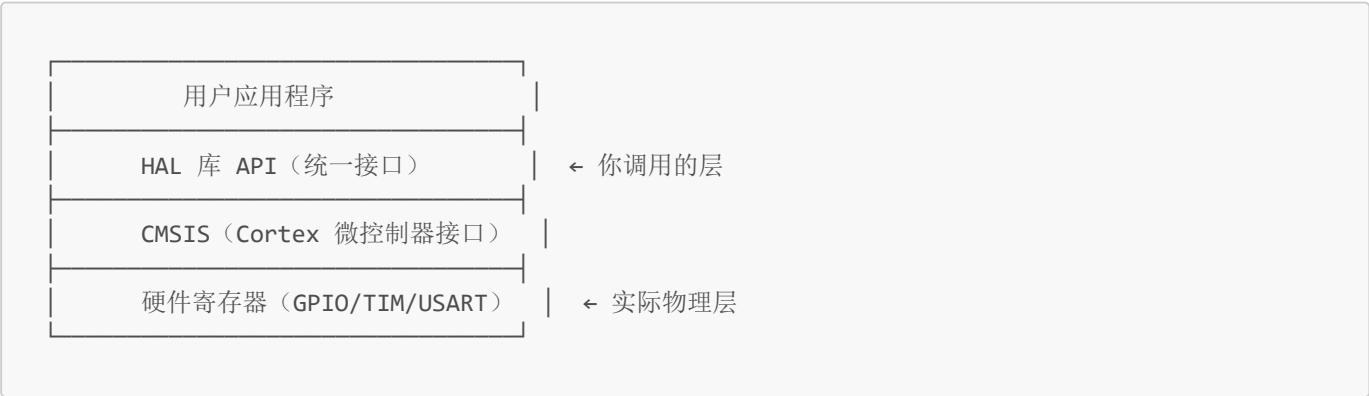
实验二：HAL 库了解

一、HAL 库概念

1.1 什么是 HAL 库？

HAL（Hardware Abstraction Layer，硬件抽象层） 是 ST 官方提供的一套固件库，它将 STM32 的硬件寄存器操作封装成统一的 API 函数，让开发者不需要直接操作寄存器即可使用外设。

核心思想：屏蔽底层硬件差异，提供统一接口。同一套 HAL API 在不同 STM32 型号间基本通用（F1/F4/F7/H7 等），移植成本低。



1.2 HAL 库的特点

优点	缺点
可移植性强，跨芯片型号迁移方便	代码体积较大（API 封装层厚）
开发效率高，API 命名规范一致	执行效率略低于直接操作寄存器
中断/DMA 处理机制完善（回调函数）	对初学者隐藏了寄存器细节
官方维护，Bug 修复持续更新	部分 API 设计过于通用，不够精简
配合 STM32CubeMX 图形化配置，可自动生成代码 —	

二、三大库对比：标准库 vs HAL 库 vs LL 库

对比维度	标准库 (StdPeriph)	HAL 库	LL 库 (Low Layer)
推出时间	2007 年（最早）	2014 年	2016 年
抽象程度	中等	高	低（接近寄存器）
代码体积	中等	大（~50KB+）	小（~10KB）
执行效率	中等	较低	高

对比维度	标准库 (StdPeriph)	HAL 库	LL 库 (Low Layer)
移植性	较差（不同系列需修改）	好（跨系列通用）	好
API 风格	<code>GPIO_SetBits(GPIOA, GPIO_Pin_0)</code>	<code>HAL_GPIO_WritePin(GPIOA, GPIO_PIN_0, GPIO_PIN_SET)</code>	<code>LL_GPIO_SetOutputPin(GPIOA, LL_GPIO_PIN_0)</code>
中断处理	需手动写中断函数体	统一回调机制	需手动写中断函数体
官方状态	已停止维护	主流推荐	保持更新
适用场景	旧项目维护	快速开发、跨平台项目	对性能/体积有极致要求

三者的互补关系

HAL 库和 LL 库可以在同一工程中混用：

- 初始化阶段用 HAL（方便配置）
- 高频调用的核心代码用 LL（追求效率）
- 例如：串口初始化用 `HAL_UART_Init()`，发送数据用 `LL_USART_TransmitData8()`

执行效率： 直接操作寄存器 > LL 库 > 标准库 > HAL 库
开发效率： HAL 库 > 标准库 > LL 库 > 直接操作寄存器
代码体积： 直接操作寄存器 < LL 库 < 标准库 < HAL 库

三、CMSIS 标准架构

3.1 什么是 CMSIS？

CMSIS（Cortex Microcontroller Software Interface Standard） 是 ARM 公司制定的 Cortex-M 系列微控制器软件接口标准，目的是让不同芯片厂商的 Cortex-M 芯片都能使用统一的软件接口。

3.2 CMSIS 的五个核心组件

- CMSIS 结构
- └ CMSIS-Core ← 核心外设访问和寄存器定义
 - └ CMSIS-DSP ← 数字信号处理库
 - └ CMSIS-RTOS ← 实时操作系统接口
 - └ CMSIS-Driver ← 外设驱动通用接口
 - └ CMSIS-SVD ← 系统视图描述（调试用）

3.3 工程中的两个关键文件夹

工程中通常有两个 CMSIS 相关的文件夹：

Device 文件夹（芯片相关）

```
Device/
└─ ST/
   └─ STM32F1xx/
      ├── Include/
      │   ├── stm32f103xb.h      ← 芯片寄存器地址映射、外设基地址、位定义
      │   ├── stm32f1xx.h       ← 包含所有 F1 系列的头文件选择
      │   └─ system_stm32f1xx.h ← 系统时钟配置声明
      └─ Source/
          └─ system_stm32f1xx.c ← 系统时钟初始化实现（配置 HCLK/PCLK）
```

- **stm32f103xb.h**: 定义了所有外设寄存器的地址和位段，比如 **GPIOA_BASE** 基地址 → 各寄存器偏移 → 每个位的作用。
- **system_stm32f1xx.c**: **SystemInit()** 函数，上电后最先执行，初始化时钟系统。

Include 文件夹（内核通用）

```
Include/
├─ core_cm3.h      ← Cortex-M3 内核寄存器定义（NVIC/SCB/SysTick）
├─ cmsis_compiler.h ← 编译器适配（Keil/IAR/GCC 宏差异）
├─ cmsis_version.h ← CMSIS 版本号
└─ mpu_armv7.h     ← 内存保护单元（MPU）定义
```

- **core_cm3.h**: 包含 SysTick、NVIC、SCB 等内核外设的结构体定义，不依赖具体芯片型号。

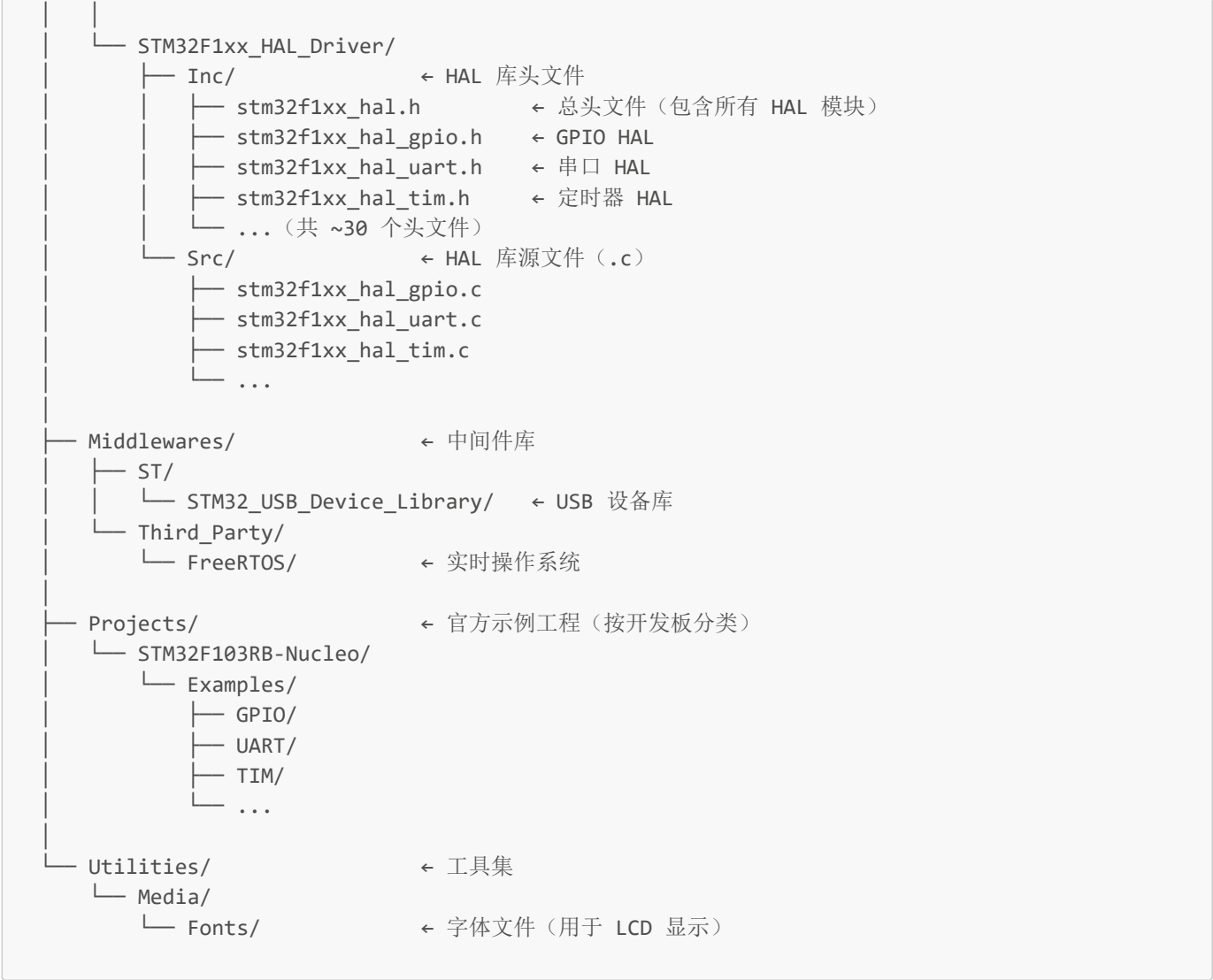
3.4 CMSIS 在工程中的位置

```
你的工程
├─ startup_stm32f103xb.s ← 启动文件（汇编），定义向量表
├─ system_stm32f1xx.c    ← CMSIS-Core: 系统时钟初始化
├─ core_cm3.h            ← CMSIS-Core: 内核寄存器
├─ stm32f103xb.h         ← CMSIS-Device: 芯片寄存器
├─ stm32f1xx_hal_conf.h  ← HAL 库配置文件（裁剪用）
└─ main.c                ← 你的代码由此开始
```

四、STM32Cube 固件包目录结构

以 **STM32Cube_FW_F1_V1.8.0** 为例：

```
STM32Cube_FW_F1_V1.8.0/
├─ Drivers/
│   ├── CMSIS/
│   │   ├── Core/      ← CMSIS-Core 内核头文件（core_cm3.h 等）
│   │   ├── Core_A/    ← Cortex-A 内核（用不到）
│   │   ├── DSP_Lib/   ← CMSIS-DSP 数字信号处理库
│   │   ├── Device/ST/STM32F1xx/ ← 芯片相关文件
│   │   │   ├── Include/ ← stm32f103xb.h, stm32f1xx.h, system_stm32f1xx.h
│   │   │   └─ Source/Templates/ ← system_stm32f1xx.c, 启动文件(.s)
│   │   ├── Include/   ← CMSIS 通用头文件
│   │   └─ RTOS2/      ← CMSIS-RTOS2 接口文件
```



新建工程时你需要拷贝的文件

源路径 (在固件包内)	拷贝到工程的位置	作用
Drivers/CMSIS/Device/ST/STM32F1xx/Source/Templates/gcc/startup_stm32f103xb.s (或 arm/)	Startup/	启动文件
Drivers/CMSIS/Device/ST/STM32F1xx/Source/Templates/system_stm32f1xx.c	User/	系统时钟初始化
Drivers/CMSIS/Device/ST/STM32F1xx/Include/ 下所有头文件	User/	芯片寄存器定义
Drivers/CMSIS/Include/ 下所有头文件	User/	CMSIS 内核定义
Drivers/STM32F1xx_HAL_Driver/Inc/ 下所有头文件	Drivers/Inc/	HAL 库头文件

源路径 (在固件包内)	拷贝到工程的位置	作用
Drivers/STM32F1xx_HAL_Driver/Src/ 下所有 .c 文件	Drivers/Src/	HAL 库源文件
Projects/.../Templates/stm32f1xx_hal_conf.h	User/	HAL 配置文件 (决定启用哪些模块)

stm32f1xx_hal_conf.h 的作用

这是 HAL 库的**裁剪配置文件**，通过 `#define HAL_xxx_MODULE_ENABLED` 宏来控制编译哪些外设模块：

```
#define HAL_MODULE_ENABLED           // 必须启用
#define HAL_GPIO_MODULE_ENABLED     // 启用 GPIO
#define HAL_UART_MODULE_ENABLED     // 启用串口
// #define HAL_SPI_MODULE_ENABLED   // 注释掉 = 不编译 SPI，减小固件体积
```

五、关键点总结

- 1. **HAL 库** = ST 官方的高级抽象 API，配合 CubeMX 开发效率最高，是本项目使用的方式。
- 2. **标准库已淘汰**，新工程不再使用；LL 库用于对性能敏感的场景。
- 3. **CMSIS** 是 ARM 标准，分两层：Core（内核通用）+ Device（芯片特定）。
- 4. **固件包四大目录**：Drivers（驱动）、Middlewares（中间件）、Projects（示例）、Utilities（工具）。
- 5. **建工程核心动作**：从固件包拷贝 CMSIS + HAL 驱动文件 → 配置头文件路径 → 编写 main.c 调用 HAL API。